

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science
Semester	V
Subject Code & Name	CS3807 – Deep Learning Laboratory
Academic Year	2026–27
Student Name	A. Priyankaa
Roll Number	24011101001

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image pre-processing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Source Code

The source code and implementation are available at the following repository:

GitHub Repository

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.

4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

7. Results

Best Hyperparameters

Hidden Layers	2
Hidden Neurons	256
Learning Rate	0.001
Batch Size	32
Optimizer	Adam
Activation Function	Sigmoid
Epochs	30
Dropout	0.0
Cross-validation Accuracy	89.75%
Testing Accuracy	89.02%

Performance Comparison

Metric	Baseline	Optimized
Accuracy	88.73%	89.02%
Precision	88.84%	89.33%
Recall	88.73%	89.02%
F1-score	88.67%	89.10%
Training Time	90s	4135.68 s (68.93 min)

8. Plots with Inference

8.1 Sample Images

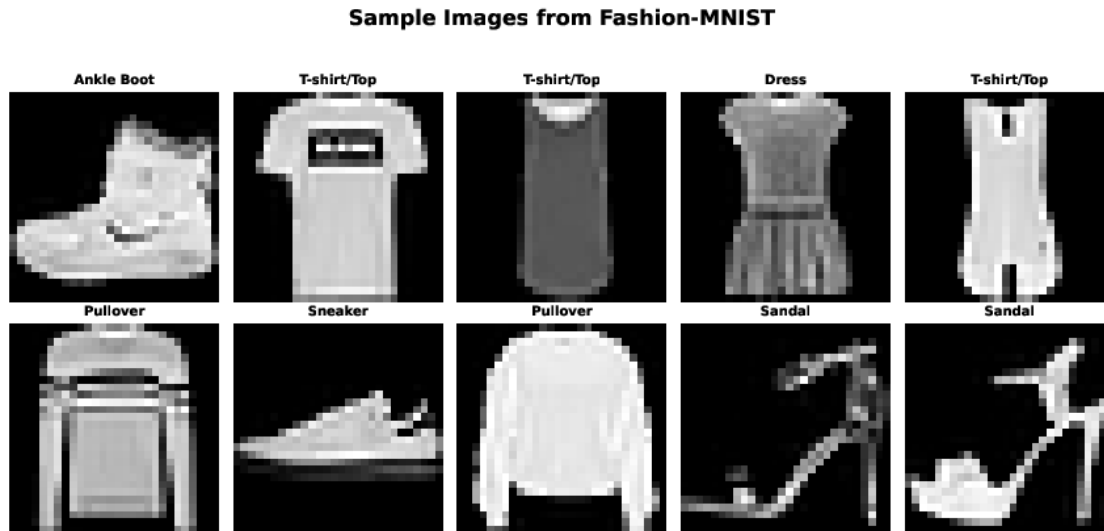


Figure 1: Sample images from the dataset.

Explanation:

The figure displays sample images from all 10 clothing categories in the Fashion-MNIST dataset.

8.2 Class Distribution

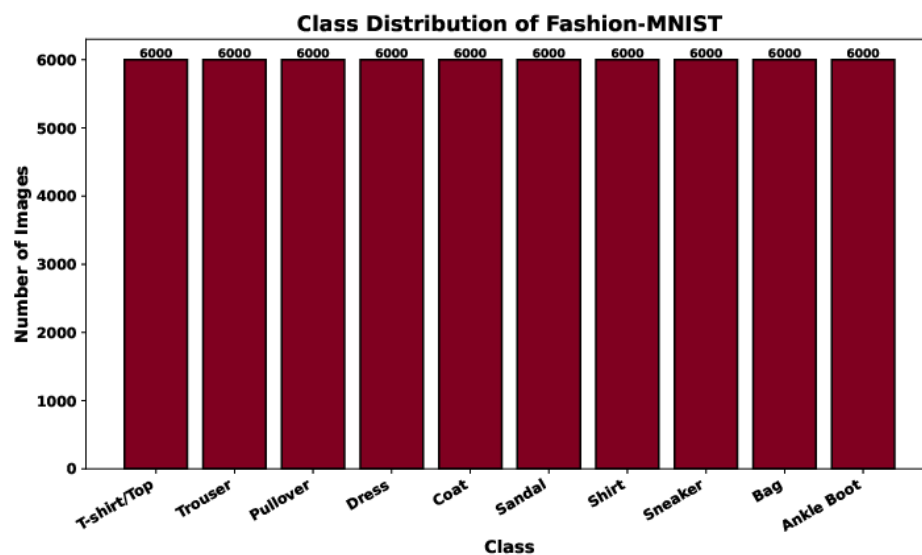


Figure 2: Distribution of images of the ten Fashion-MNIST classes.

Inference:

The bar chart shows that all ten classes contains 6000 images. This provides a balanced dataset and allows the model to learn each category without any bias.

8.3 Training and Validation Accuracy

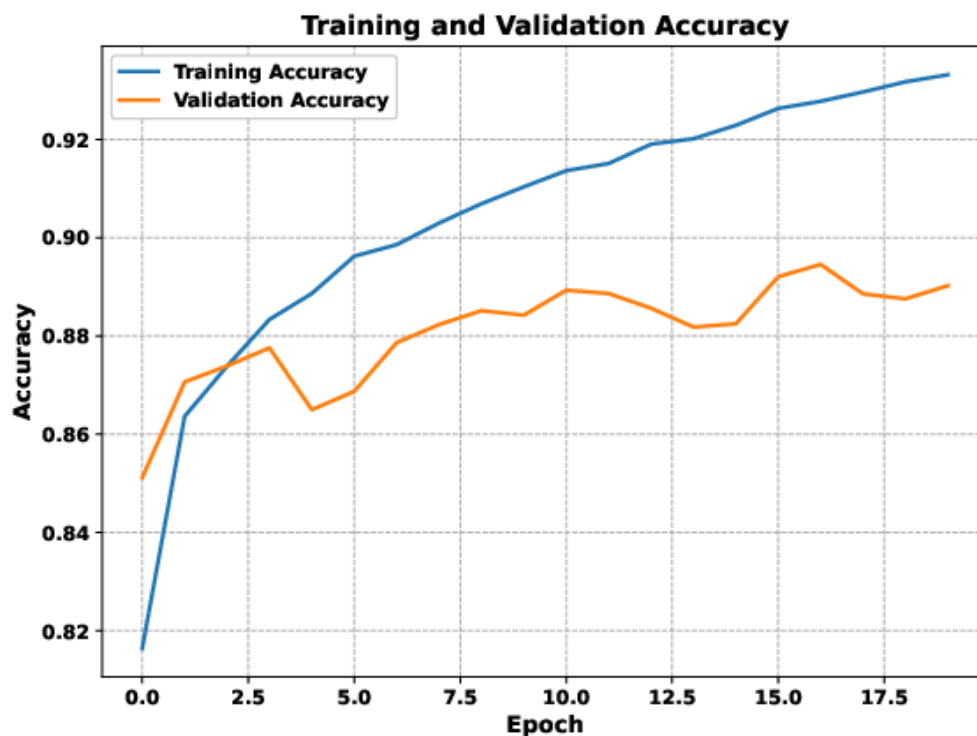


Figure 3: Training and validation accuracy of the MLP model over epochs.

Inference:

The training accuracy steadily improved throughout the training process from 81.6% to 93.4%. The validation accuracy rises from 85.1% to 89.0% with small fluctuations. The gap between the two curves widens during the later epochs indicating slight overfitting.

8.4 Training and Validation Loss



Figure 4: Training and validation loss of the MLP model over epochs.

Inference:

The training loss decreased steadily during training so this shows that the model was learning the training data well. The validation loss decreases from 0.41 to 0.32 during the initial epochs but later fluctuates between 0.32 and 0.37. This suggests overfitting.

8.5 Confusion Matrix

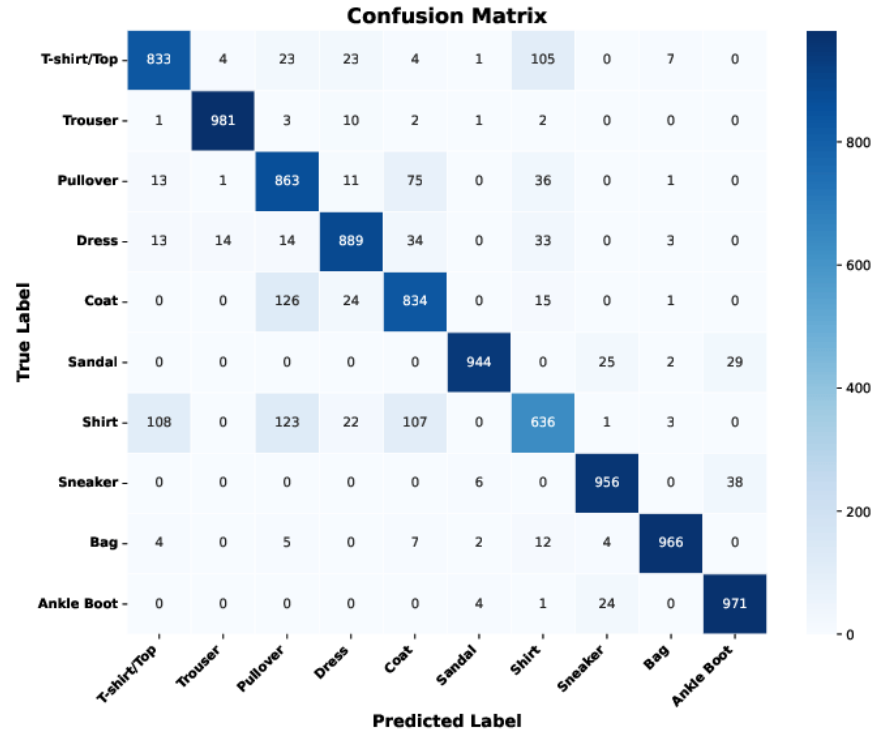


Figure 5: Confusion matrix of the baseline MLP model on the test dataset.

Inference:

The confusion matrix shows that most of the images are correctly classified. Categories such as trouser, sneaker and bag have high classification accuracy while most misclassifications occur between visually similar classes such as t-shirt and shirt, and pullover and coat.

8.6 Hyperparameter Search Results

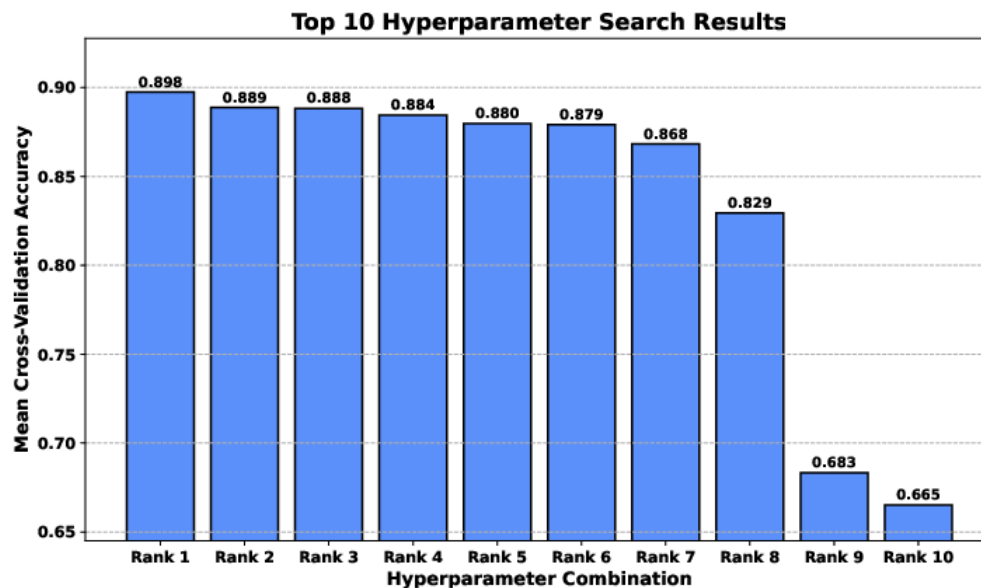


Figure 6: Performance of the top hyperparameter combinations obtained using Randomized Search.

Inference:

The best hyperparameter configuration achieved an accuracy of 89.8%. Most of the other combinations also gave similar results whereas the lower ranked combinations achieved lower accuracies. This shows that choosing suitable hyperparameters helps improve the performance of the model.

8.7 Baseline vs Optimized Model Comparison

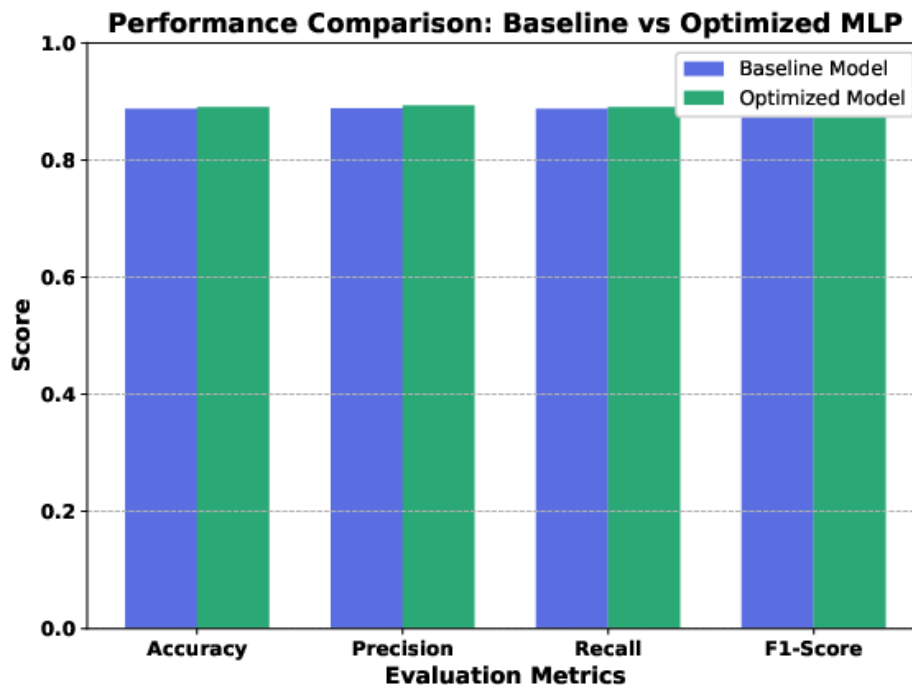


Figure 7: Comparison of the baseline and optimized MLP model performance.

Inference:

The optimized MLP model achieved better performance than the baseline model on all the evaluation metrics. This shows that choosing suitable hyperparameters helps improve the performance of the model.

9. Discussion

1. Which optimization method was used?

Using Randomized Search with 5 fold cross validation. Different hyperparameter combinations were tested and the combination with the highest accuracy was chosen.

2. Which hyperparameters were selected?

The best hyperparameters were:

- Hidden Layers: 2
- Hidden Neurons: 256
- Learning Rate: 0.001
- Optimizer: Adam
- Activation Function: Sigmoid
- Batch Size: 32

- Epochs: 30
- Dropout Rate: 0.0

3. Did optimization improve performance?

Yes. The optimized model gave slightly better results than the baseline model with the testing accuracy increasing from 88.73

4. Which hyperparameter had the greatest impact?

The combination of Adam optimizer, the learning rate of 0.001 and it had two hidden layers with 256 neurons and used the sigmoid activation function.

5. Compare Grid Search and Randomized Search.

Grid Search tries all possible hyperparameter combinations and finds the best one but takes more time. Randomized Search speeds up by testing only a few random combinations which produces similar results in lesser time.

6. Recommend the best MLP configuration. The best model was with two hidden layers of 256 neurons. It used the Adam optimizer with a learning rate of 0.001. The sigmoid activation function was used with a batch size of 32. The model was trained for 30 epochs without dropout. It gave the highest cross validation accuracy and also gave the best results on the test dataset.

10. Additional Tasks

- 1. Code the Perceptron Learning Algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging.**

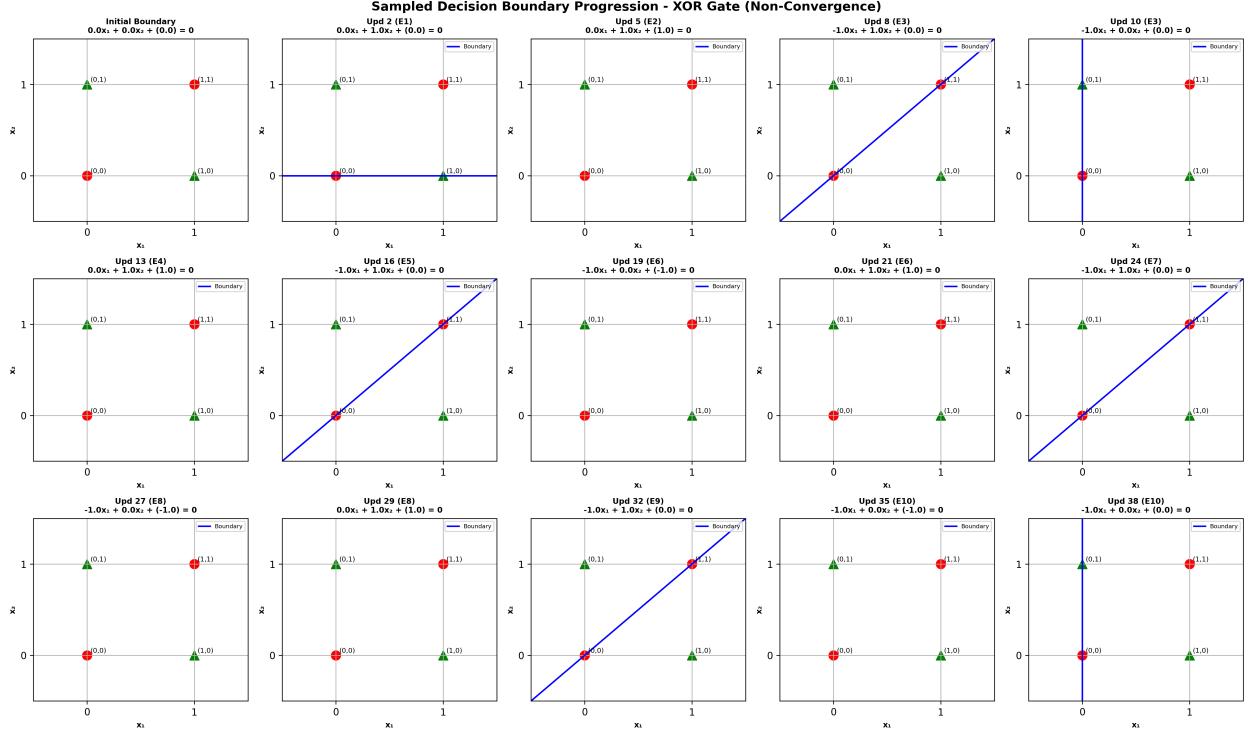


Figure 8: Decision boundary progression for the XOR gate.

Table 1: Weight Update History for the XOR Gate

Update	Input	Target	Prediction	Weights	Bias
1	[0,0]	0	1	[0,0]	-1
2	[0,1]	1	0	[0,1]	0
3	[1,1]	0	1	[-1,0]	-1
4	[0,1]	1	0	[-1,1]	0
5	[1,0]	1	0	[0,1]	1
6	[1,1]	0	1	[-1,0]	0

Inference:

The perceptron didn't converge while learning the XOR gate. The decision boundary kept shifting and the weights kept cycling instead of reaching stable values. The training accuracy remained at 50% showing that the model could not classify all four input combinations correctly. It is because XOR gate is not linearly separable so a straight line cannot separate the classes. So a Multi Layer Perceptron with hidden layers is needed.

11. Conclusion

Multi Layer Perceptron is implemented on multi class image classification of Fashion-MNIST dataset. Baseline model was trained, evaluated and hyperparameter tuning was performed using Randomized Search along with 5 fold Cross Validation. This model achieved better evaluation metrics than the baseline model. This shows that selecting suitable hyper parameters helps improve the performance of model.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.